

AFRILANG Stdlib

std/m/interpolate2

Français. Bibliothèque AFRILANG 'std/m/interpolate2' (niveau medium, catégorie medium). Elle expose 4 fonction(s) : lerp, lerpList, inverseLerp, remap.

```
'import "std/m/interpolate2"' · 'use interpolate2'
```

```
- 'lerp(a number, b number, t number) → number' - 'lerpList(a list  
number, b list number, t number) → list number' - 'inverseLerp(a  
number, b number, v number) → number' - 'remap(v number, inLo  
number, inHi number, outLo number, outHi number) → number'
```

English. AFRILANG library 'std/m/interpolate2' (tier medium, category medium). Exports 4 function(s): lerp, lerpList, inverseLerp, remap.

```
'import "std/m/interpolate2"' · 'use interpolate2'
```

```
- 'lerp(a number, b number, t number) → number' - 'lerpList(a list num-  
ber, b list number, t number) → list number' - 'inverseLerp(a number,  
b number, v number) → number' - 'remap(v number, inLo number, inHi  
number, outLo number, outHi number) → number'
```

Catégorie / Category Modules medium (m/)

Niveau / Tier medium

Fonctions / Functions 4

June 16, 2026

Contents

Français

1. Vue d'ensemble

Bibliothèque AFRILANG 'std/m/interpolate2' (niveau medium, catégorie medium). Elle expose 4 fonction(s) : lerp, lerpList, inverseLerp, remap.

'import "std/m/interpolate2"' · 'use interpolate2'

```
- `lerp(a number, b number, t number) → number`
- `lerpList(a list number, b list number, t number) → list number`
- `inverseLerp(a number, b number, v number) → number`
- `remap(v number, inLo number, inHi number, outLo number, outHi number) → number`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/m/interpolate2"
use interpolate2
```

Niveau : medium · Catégorie : Modules medium (m/)
Bibliothèques intermédiaires – import std/m.../

3. Référence API

- lerp(a number, b number, t number) → number•
- lerpList(a list number, b list number, t number) → list•
- inverseLerp(a number, b number, v number) → number•
- remap(v number, inLo number, inHi number, outLo number, outHi number) → number

4. Exemples d'utilisation

```
import "std/m/interpolate2"
use interpolate2
```

```
create result = lerp(/* args */)
say result
```

```
create result = lerpList(/* args */)
say result
```

```
create result = inverseLerp(/* args */)
say result
```

```
create result = remap(/* args */)
say result
```

5. Bonnes pratiques

- Préférez 'import "std/m/interpolate2"' en tête de fichier.
- Lancez 'afirang check' avant de livrer.
- Combinez avec les modules stdlib de la même catégorie.
- Voir /stdlib/ pour le source live et les démos playground.
- Signalez les problèmes sur GitHub si une export manque.

6. Annexe — source

module interpolate2

```
function lerp(a number, b number, t number) returns number  
end
```

```
function lerpList(a list number, b list number, t number) returns list number  
end
```

```
function inverseLerp(a number, b number, v number) returns number  
end
```

```
function remap(v number, inLo number, inHi number, outLo number, outHi number)  
  returns number  
end  
end
```

English

1. Overview

AFRILANG library 'std/m/interpolate2' (tier medium, category medium). Exports 4 function(s): lerp, lerpList, inverseLerp, remap.

'import "std/m/interpolate2"' · 'use interpolate2'

```
- `lerp(a number, b number, t number) → number`
- `lerpList(a list number, b list number, t number) → list number`
- `inverseLerp(a number, b number, v number) → number`
- `remap(v number, inLo number, inHi number, outLo number, outHi number) → number`
```

2. Installation & import

Add the module to your program:

```
import "std/m/interpolate2"
use interpolate2
```

Tier: medium · Category: Medium modules (m/)
Intermediate libraries – import std/m.../

3. API reference

- lerp(a number, b number, t number) → number•
- lerpList(a list number, b list number, t number) → list•
- inverseLerp(a number, b number, v number) → number•
- remap(v number, inLo number, inHi number, outLo number, outHi number) → number

4. Usage examples

```
import "std/m/interpolate2"
use interpolate2
```

```
create result = lerp(/* args */)
say result
```

```
create result = lerpList(/* args */)
say result
```

```
create result = inverseLerp(/* args */)
say result
```

```
create result = remap(/* args */)
say result
```

5. Best practices

- Prefer `import "std/m/interpolate2"` at the top of your file.
- Run `afrilang check` before shipping.
- Combine with related stdlib modules from the same category.
- See /stdlib/ for the live source and playground demos.
- Report issues on GitHub if an export is missing or undocumented.

6. Appendix — source

module interpolate2

```
function lerp(a number, b number, t number) returns number
end
```

```
function lerpList(a list number, b list number, t number) returns list number
end
```

```
function inverseLerp(a number, b number, v number) returns number
end
```

```
function remap(v number, inLo number, inHi number, outLo number, outHi number)
  returns number
end
end
```

Référence rapide / Quick reference

- Import : `import "std/m/interpolate2"`
- Vérification : `afrilang check`
- Documentation : <https://afrilang.dev/stdlib/std/m/interpolate2/>

Source (extrait)

```
module interpolate2

function lerp(a number, b number, t number) returns number
end

function lerpList(a list number, b list number, t number) returns list number
end

function inverseLerp(a number, b number, v number) returns number
end

function remap(v number, inLo number, inHi number, outLo number, outHi number)
  returns number
end
end
```